

SQL Refactoring: Alt Sorgudan CTE Yapısına Geçiş

Veri biliminde kodun okunabilirliği, iş birliği yapılan diğer ekip üyeleri ve ileride kodu tekrar inceleyecek olan sizler için hayati önem taşır. Aşağıdaki örnekte, her kategorideki ürünlerin ortalama fiyatını hesaplayıp, sadece ortalama fiyatın üzerinde kalan ürünleri listeleyen bir senaryoyu karşılaştırıyoruz.

1. Karmaşık ve İç İç Geçmiş Yapı (Subquery)

Bu yöntemde kodun ne yaptığı, parantezlerin derinliği arttıkça anlaşılmaz hale gelir. Hata ayıklamak (debug) oldukça zordur.

```
-- OKUNMASI ZOR YAPI (NESTED SUBQUERY)
SELECT s.product_name, s.price, s.category_avg
FROM (
    SELECT p1.product_name, p1.price,
           (SELECT AVG(p2.price)
            FROM products p2
            WHERE p2.category = p1.category) AS category_avg
    FROM products p1
) AS s
WHERE s.price > s.category_avg;
```

2. Temiz ve Modüler Yapı (CTE)

Bu yöntemde önce "ne hazırlayacağımızı" tanımlarız (WITH bloğu), sonra bu hazırlığı ana sorgumuzda kullanırız. Kod, yukarıdan aşağıya bir hikaye gibi okunur.

```
-- TEMİZ VE MODÜLER YAPI (CTE)
WITH CategoryAverages AS (
    SELECT category, AVG(price) AS avg_price
    FROM products
    GROUP BY category
)
SELECT p.product_name, p.price, ca.avg_price
FROM products p
JOIN CategoryAverages ca ON p.category = ca.category
WHERE p.price > ca.avg_price;
```

Neden CTE Tercih Edilmeli?

- **Mantıksal Ayrım:** Veri hazırlama süreci ile ana analiz süreci birbirinden ayrılır.
- **Tekrar Kullanılabilirlik:** CategoryAverages tablosunu aynı sorgu içinde farklı yerlerde tekrar tekrar çağırabilirsiniz.
- **Hata Ayıklama:** CTE kısmını tek başına çalıştırıp sonuçların doğruluğunu kontrol etmek, iç içe geçmiş bir alt sorguyu parçalamaktan çok daha kolaydır.